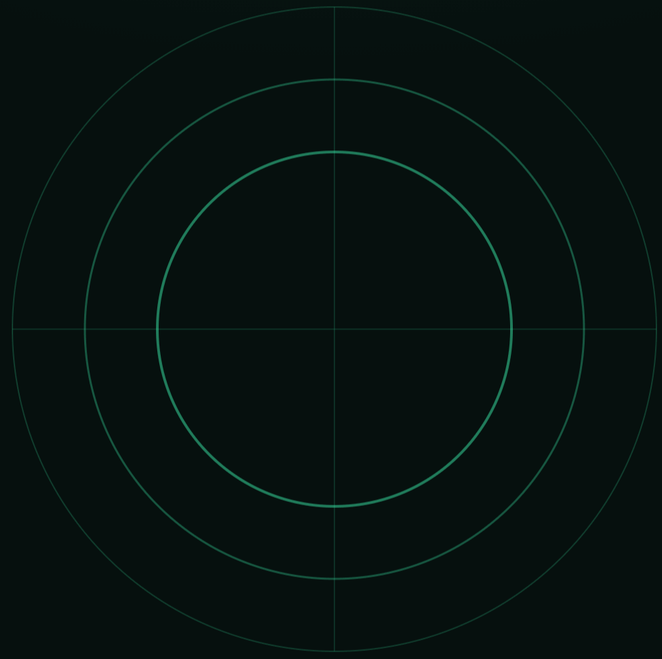
PATHFINDER · AI — DATA MINING PROJECT

Pathfinder AI

An AI-based career recommendation system that turns skills, interests & education into personalized, explainable guidance.



TERM
Spring 2026

PRESENTED BY
Zain Irshedat

PIPELINE
Mining · ML · Recommender

Understanding the data before modeling.

200	32	8	0
CANDIDATE PROFILES	CAREER CATEGORIES	RAW FEATURES	MISSING VALUES

CAREER DISTRIBUTION · TOP 10 OF 32

+ 24 MORE CATEGORIES — A LONG TAIL OF ~4 PROFILES EACH

AGE DISTRIBUTION

MEAN 30 · MEDIAN 29 · RANGE 22–41

MOST FREQUENT SKILLS & INTERESTS

Python · Data Analysis · SQL · Machine Learning · JavaScript · Excel · Design · Finance

SKILLS & INTERESTS ARE THE MOST INFORMATIVE SIGNALS — AND NO VALUES ARE MISSING.

Making the data machine-ready.

Algorithms can't read raw text or labels. Three transforms turn every profile into a clean numeric vector.

01 · CATEGORICAL

Label Encoding

Education levels become ordered integers the model can compare.

`"Bachelor" → 1 · "Master" → 2`

02 · NUMERIC

Standard Scaling

Age is normalized to zero mean & unit variance so it can't dominate.

`μ = 0 · σ = 1`

03 · TEXT

TF-IDF Vectorization

Skills & interests become weighted vectors — rare, career-specific terms count more.

`text → weighted vector`

INPUT
Raw profiles

→

3 TRANSFORMS
Encode · Scale · Vectorize

→

OUTPUT
102-feature matrix · ML-ready

EVERY PROFILE BECOMES A SINGLE 102-DIMENSIONAL FEATURE VECTOR.

Less noise, same signal.

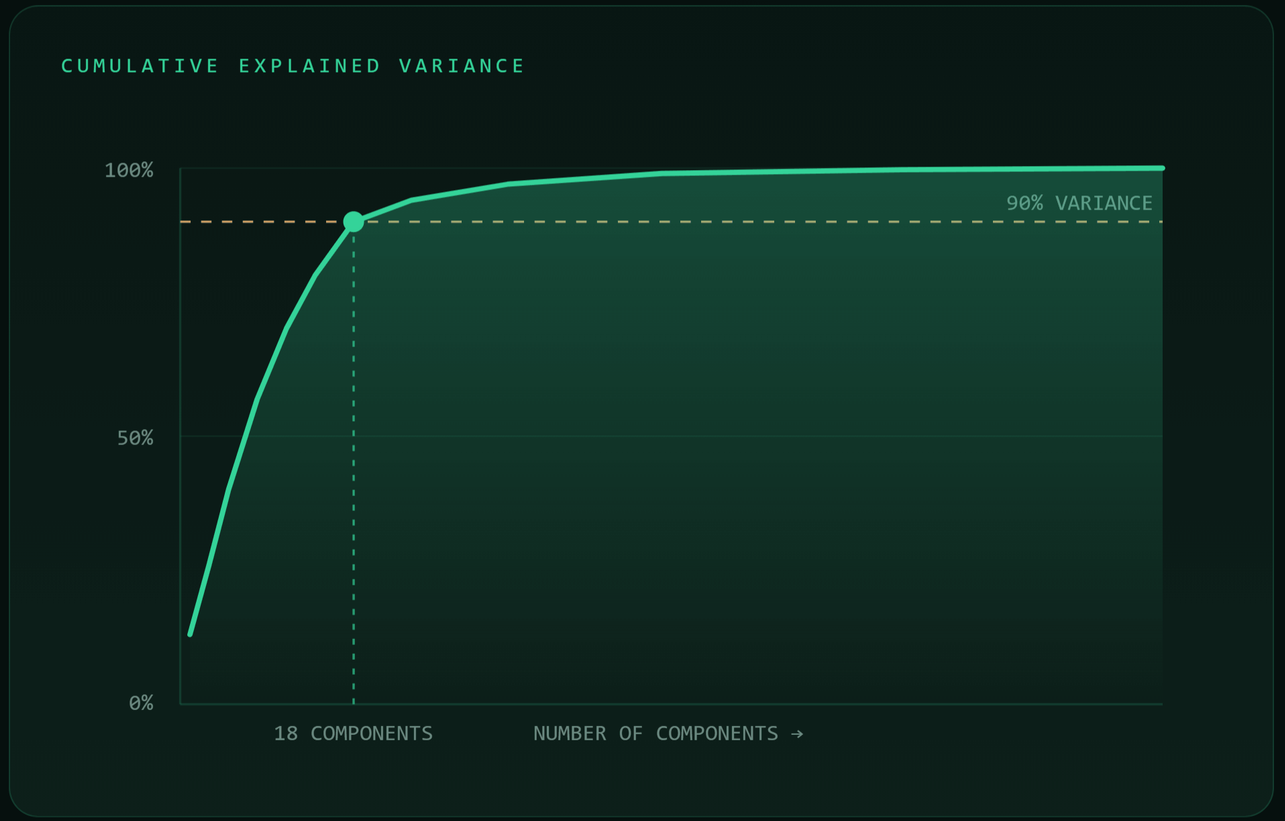
Principal Component Analysis compresses 102 correlated features into a compact set that keeps almost all the information.

102 original features after preprocessing

↓ PCA project onto principal components

18 principal components retained

~90% of the original variance preserved



AN ~82% SMALLER FEATURE SPACE WITH VIRTUALLY NO INFORMATION LOST.

Looking for natural groups.

Unsupervised K-Means grouped candidates purely by profile similarity — a diagnostic lens on the data's structure. Optimal K was chosen with the elbow method & silhouette score.

K = 2
OPTIMAL CLUSTERS

0.29
SILHOUETTE SCORE

WHY IT ISN'T THE FINAL ENGINE

A low silhouette means the clusters don't cleanly match the 32 real career labels. Clustering stayed an exploratory tool, not the recommender.

EXPLORATORY PATTERN DISCOVERY — NOT USED FOR FINAL RECOMMENDATIONS.

Apriori, built from scratch.

Implemented manually in Python — no library shortcuts. Apriori surfaces skill & interest combinations that frequently occur together, making recommendations explainable.

DISCOVERED ASSOCIATION RULES

Python → Machine Learning	CONF 0.86
HTML, CSS → JavaScript	CONF 0.91
Data Analysis → SQL	CONF 0.78
Financial Analysis → Excel	CONF 0.74

RULES ILLUSTRATE PATTERN STRUCTURE FROM THE TRANSACTION SET

HOW IT WORKS

Count frequent itemsets → prune below minimum support → generate rules above a confidence threshold.

WHY IT MATTERS

Recommendations can point to real, recurring patterns in the data — not a black box.

FREQUENT SKILL COMBINATIONS BECOME HUMAN-READABLE REASONS.

The core: matching by similarity.

Our first goal was a system that truly understands the similarity between candidates — using cosine similarity on the TF-IDF skill vectors.

INPUT
User profile skills + interests

→

VECTORIZE
TF-IDF weighted vectors

→

COSINE
Angle vs every candidate / centroid

→

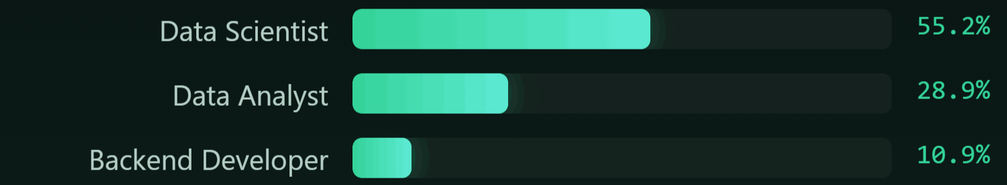
OUTPUT
Top-5 careers + match %

WHY COSINE SIMILARITY?

It doesn't just *count* shared skills — it measures their direction & quality by the angle between a new profile and every candidate. A closer angle means a stronger match.

A FLEXIBLE RECOMMENDER — NOT A RIGID SINGLE-LABEL CLASSIFIER

LIVE DEMO · PYTHON · MACHINE LEARNING

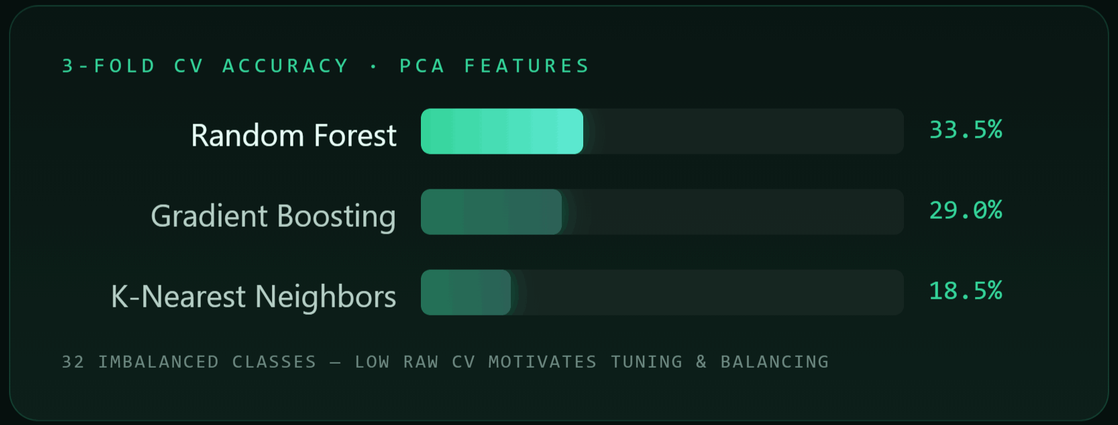


HTML · CSS → FRONT-END DEVELOPER 99.7%

TOP-5 ALTERNATIVE CAREERS WITH A CLEAR MATCH PERCENTAGE.

Choosing a classifier.

Next we built an ML classifier to predict the primary career directly. Three models were compared with 3-fold cross-validation on the PCA features.



CARRIED FORWARD

Random Forest

Clear winner of the three — most accurate and most stable across folds.

Next → tune hyperparameters, engineer features & balance classes.

Tuning the winner.

We trained the final Random Forest on the full TF-IDF matrix, tuned for the complexity of the text data.

`n_estimators = 400` `max_depth = 20` `class_weight = balanced`

94.4%
TRAIN ACCURACY
Fits the training data closely.

74.8%
TEST ACCURACY
Held-out generalization across 32 careers.

88.1%
TOP-3 ACCURACY ★
The key metric for a recommender — is the real career in our top 3?

Because this is a recommender, Top-3 accuracy matters most — it checks whether the user's actual career appears among our three suggestions. 88.1% of the time, it does.

TOP-3 ACCURACY IS THE METRIC THAT MATTERS FOR RECOMMENDATIONS.

Closing the overfitting gap.

We added Skills Count & Interests Count features (→ 104 total), then applied SMOTE to balance the classes.

SMOTE · CLASS BALANCING

200 → 672

IMBALANCED → BALANCED

Some of the 32 careers had only ~3 examples. SMOTE (k = 2) synthesizes minority classes so the model isn't biased toward the common careers.

94.4%	77.0%	87.4%
TRAIN ACC	TEST ACC	TOP-3 ACC

OVERFITTING GAP

19.6% → **17.4%**

Test accuracy rose to 77.0% while the train–test gap narrowed.

BALANCED CLASSES AND RICHER FEATURES — A TIGHTER, FAIRER MODEL.

Pushing further — and what won.

We tested multi-hot encoding, XGBoost, and a soft-voting ensemble (RF + XGB) to see if accuracy could climb higher.

MODEL	TEST ACCURACY	TEST	TOP-3
RF + TF-IDF · baseline ★		**77.8%**	**87.4%**
RF + Multi-hot		71.9%	87.4%
XGBoost + Multi-hot		71.9%	83.7%
Ensemble · RF + XGB		70.4%	84.4%

CONCLUSION

Random Forest + TF-IDF stayed the strongest and most stable approach. TF-IDF tokenization actually helped the model discover hidden connections between shared words across skills — outperforming rigid single-term encodings.

Thank you.

I'd be happy to answer any questions.

PRESENTED BY
Zain Irshedat

PROJECT
AI Career Recommendation System